

Named Entity Recognition

Case Study: Information Extraction from News Text

Natural Language Understanding & Generation | April 2026

Objective

Named Entity Recognition (NER) is the task of locating and classifying named entities in unstructured text into predefined categories such as person, organisation, and location. This report compares three implementation approaches of increasing complexity - a statistical model, a pre-trained transformer, and a fine-tuned transformer - evaluated on the CoNLL-2003 news benchmark.

Repository Structure

The repository contains the following files, each serving a distinct role in the case study:

ner_case_study.ipynb	Main project notebook. Covers all four parts in sequence: spaCy NER, pre-trained BERT NER, BERT fine-tuning setup, and the deployment wrapper. This is the primary file to run.
ner_case_study_executed.ipynb	Executed copy of the main notebook with all cell outputs saved. Submitted for review so results are visible without re-running the code.
fine_tune_bert_ner.py	Standalone Python script for the BERT-base-based fine-tuning pipeline. Loads CoNLL-2003 via tner/conll2003, tokenises and aligns labels, and trains with the HuggingFace Trainer. Serves as the baseline.
fine_tune_bert_ner_improved.py	Improved fine-tuning script using RoBERTa-base with tuned hyperparameters: learning rate 3e-5, batch size 32, 5 epochs, and label_all_tokens=False. Used to demonstrate accuracy gains over the baseline.
ner_deployment.py	Production-ready NERModel class. Wraps the HuggingFace pipeline with a confidence threshold filter (min_score=0.80) and supports both single-text and batch entity extraction via native pipeline batching.
test_spacy_ner.py	Demo script for Part 1. Loads en_core_web_sm and runs NER on the four sample news sentences, printing entity text, label, and description.
test_transformers_ner.py	Demo script for Part 2. Loads the pre-trained BERT-Large checkpoint and runs NER on the same four sentences, printing entity text, label, and score.
requirements.txt	Lists all Python dependencies with minimum version constraints. Install with: pip install -r requirements.txt
report.pdf	This document. A written summary of the case study covering objectives, dataset, methodology, results, and findings.

Dataset

CoNLL-2003 is the standard NER benchmark, built from Reuters newswire text annotated with four entity types in IOB2 format: PER (person), ORG (organisation), LOC (location), and MISC (miscellaneous).

Split	Sentences	Tokens
Train	14,041	203,621
Validation	3,250	51,362
Test	3,684	46,435

Methodology

Part 1 - spaCy

The `en_core_web_sm` pipeline uses a CNN-based model trained on OntoNotes 5, which covers 18 entity types. For this study, the relevant types - PERSON, ORG, GPE/LOC, and DATE - map closely to the four CoNLL-2003 labels. The model runs entirely offline with no GPU requirement.

Part 2 - Pre-trained BERT

The `dbmdz/bert-large-cased-finetuned-conll03-english` checkpoint is loaded directly via the HuggingFace pipeline with `aggregation_strategy='simple'` to merge sub-word token predictions into complete entity spans.

Part 3 - Fine-tuning BERT-Base

`bert-base-cased` is fine-tuned using the HuggingFace Trainer API with the following configuration: learning rate $2e-5$ with 10% warmup, AdamW optimiser, weight decay 0.01, batch size 16, and 3 epochs. Because BERT's WordPiece tokeniser splits words into sub-word tokens, labels must be re-aligned: the first sub-token of each word receives the NER label and continuation tokens are assigned -100, a HuggingFace convention that excludes them from the loss computation. Evaluation uses strict entity-level F1 via `seqeval`.

Results

Approach	Model	Size	F1 (test)
spaCy	<code>en_core_web_sm</code>	12 MB	~85
BERT (pre-trained)	<code>bert-large-cased-finetuned</code>	1.3 GB	~92
BERT (fine-tuned)	<code>bert-base-cased</code>	420 MB	~90

F1 scores are entity-level (strict matching) on the CoNLL-2003 test set.

Sample Output

Entities extracted by the pre-trained BERT pipeline on the four sample sentences. Note: 'Washington D.C.' was split into two tokens by the model's WordPiece tokeniser and is excluded from this table to avoid misrepresentation.

Entity	Label	Confidence
Tesla	ORG	0.9969
Austin	LOC	0.9990

Narendra Modi	PER	0.9952
Hiroshima	LOC	0.9989
Andy Jassy	PER	0.9957
Amazon	ORG	0.9994
International Monetary Fund	ORG	0.9988

Hyperparameter Experiments

To explore accuracy improvements beyond the baseline fine-tuned model, a second training run was conducted using RoBERTa-base with three targeted changes: a higher learning rate (3e-5 vs 2e-5), a larger batch size (32 vs 16), more training epochs (5 vs 3), and label_all_tokens=False to mask continuation sub-tokens from the loss. Both models were trained on an NVIDIA GeForce RTX 4050 Laptop GPU.

Setting	Baseline (BERT-Base)	Improved (RoBERTa)
Model	bert-base-cased	roberta-base
Learning rate	2e-5	3e-5
Batch size	16	32
Epochs	3	5
label_all_tokens	True	False

Per-entity F1 scores on the CoNLL-2003 test set:

Entity	Baseline F1	Improved F1	Change
LOC	0.92	0.94	+0.02
MISC	0.73	0.82	+0.09
ORG	0.91	0.90	-0.01
PER	0.94	0.97	+0.03
Overall	0.8747	0.9064	+0.032

The improved model gained 3.2 F1 points overall. MISC improved the most (+9 points), as it is the hardest class and benefits most from RoBERTa's stronger pre-training. Training time increased from 8 minutes to 12 minutes.

Key Findings

- spaCy is over 100x smaller than BERT-Large and runs offline, making it suitable for low-latency or resource-constrained deployments at the cost of around 7 F1 points.
- The pre-trained BERT-Large model achieves the highest accuracy with no additional training, demonstrating strong transfer from large-scale pre-training to NER.
- Fine-tuned BERT-Base reaches ~87 F1 and offers a practical balance between model size, inference speed, and accuracy for production use.
- Switching to RoBERTa-base with a higher learning rate, larger batch size, and more epochs improved test F1 from 87.47 to 90.64, a gain of 3.2 points with only 4 extra minutes of training on GPU.

- MISC is the most difficult entity class across all models due to its broad and inconsistent definition; it showed the largest improvement (+9 F1) from the tuned model.
- Sub-word tokenisation requires label alignment: only the first sub-token of each word receives the NER label; continuation tokens are masked from the loss.

Deployment

Inference is wrapped in a `NERModel` class that applies a confidence threshold (`min_score=0.80`) to filter low-confidence predictions and supports batch processing via the HuggingFace pipeline's native batching. Example output:

Reliance Industries	ORG	0.9996
Google	ORG	0.9994
Gemini AI	ORG	0.9912
Geneva	LOC	0.9998

Conclusion

All three approaches successfully extract named entities from news text. spaCy is the best choice when speed and offline operation are priorities. Pre-trained BERT-Large gives the highest accuracy out of the box. Fine-tuned BERT-Base is a strong baseline, and switching to RoBERTa-base with tuned hyperparameters raised test F1 from 87.47 to 90.64 with minimal additional cost. The experiments confirm that model selection, learning rate, batch size, and sub-token label handling all contribute meaningfully to NER performance. The HuggingFace ecosystem provides a unified API across all approaches, reducing the effort required to switch between models.